Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ: «Информатика и системы управления»
КАФЕДРА: «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №15
«Параллельный ГА»

по курсу
«Методы оптимизации»

Выполнил:
студент группы ИУ9-82Б
Караник А.А.

Проверено:
Посевин Д.П.

Москва, 2025

Цель работы

Разработка и сравнение параллельного и последовательного генетических алгоритмов (ГА) для оптимизации функции Розенброка с целью ускорения сходимости и улучшения качества решения за счёт параллельных вычислений и миграции особей между подпопуляциями.

Реализация

Исходный код:

```
using Random
using Plots
using HTTP
using JSON
using Distributed
using Base.Threads: @spawn, ReentrantLock, lock
using ProgressMeter

const POPULATION_SIZE = 30
const GENERATIONS = 500
const NUM_WORKERS = 3

const DISTANCE_PARAM = 0.5
const MIGRATION_INTERVAL = 10
const MIGRATION_COUNT = 5

const Chromosome = Vector{Bool}

function evaluate_rosenbrock(x, y)
    return (1.0 - x)^2 + 100.0 * (y - x^2)^2
end

function decode_chromosome(chrom::Vector{Bool})
    half = div(length(chrom), 2)
    x_bits = chrom[1:half]
    y_bits = chrom[half+1:end]
    x = binary_to_float(x_bits, -2.0, 2.0)
    y = binary_to_float(y_bits, -1.0, 3.0)
    return x, y
end

function binary_to_float(bits::Vector{Bool}, min_val::Float64, max_val::Float64)
    intval = sum(bits[i] * 2^(length(bits)-i) for i in 1:length(bits))
    max_int = 2^length(bits) - 1
    min_val + (max_val - min_val) * intval / max_int
end

function create_random_chromosome()
    rand(Bool, 32)
end

function compute_fitness(chromo::Chromosome)
    x, y = decode_chromosome(chromo)
    -evaluate_rosenbrock(x, y)
end

function calculate_distance(a::Chromosome, b::Chromosome)
    min_val = 0
    max_val = 1
    n = length(a)
    dist = 0
```

```julia
    for i in 1:n
        dist += abs((a[i]-b[i]) / (max_val - min_val))
    end
    normalized = dist / n
    return normalized^DISTANCE_PARAM
end

function apply_mutation!(chromo::Chromosome, mutation_prob::Float64)
    for i in eachindex(chromo)
        if rand() < mutation_prob
            chromo[i] = !chromo[i]
            break
        end
    end
end

function perform_crossover(parent1::Chromosome, parent2::Chromosome)
    point = rand(2:31)
    child1 = vcat(parent1[1:point], parent2[point+1:end])
    child2 = vcat(parent2[1:point], parent1[point+1:end])
    return child1, child2
end

function select_parent(pop::Vector{Chromosome})
    a, b = rand(pop, 2)
    compute_fitness(a) > compute_fitness(b) ? a : b
end

function evolve_population!(population)
    elite_size = div(POPULATION_SIZE, 3)
    sort!(population, by=compute_fitness, rev=true)
    elite = population[1:elite_size]
    new_population = Chromosome[]

    while length(new_population) < POPULATION_SIZE - elite_size
        parent1 = select_parent(population)
        parent2 = select_parent(population)
        child1, child2 = perform_crossover(parent1, parent2)

        distance = calculate_distance(parent1, parent2)
        mutation_prob = 1.0 - distance

        apply_mutation!(child1, mutation_prob)
        apply_mutation!(child2, mutation_prob)
        push!(new_population, child1)
        if length(new_population) < POPULATION_SIZE - elite_size
            push!(new_population, child2)
        end
    end
    vcat(elite, new_population)
end

function handle_messages(ws, population, expected_messages)
    for _ in 1:expected_messages
        msg = HTTP.WebSockets.receive(ws)
        data = JSON.parse(msg)
        for raw_ind in data["individuals"]
            chromo = map(Bool, raw_ind)
            push!(population, chromo)
        end
    end
    sort!(population, by=compute_fitness, rev=true)
    resize!(population, POPULATION_SIZE)
end
```

```julia
function worker_process(id::Int, population::Vector{Chromosome},
                    time_history::Dict{Float64, Float64},
                    gen_history::Dict{Int, Float64},
                    global_best::Base.RefValue{Tuple{Float64, Float64, Float64}},
                    history_lock::ReentrantLock,
                    start_time::Float64)
    for generation in 1:GENERATIONS
        current_time = (time() - start_time) * 1000

        if generation % MIGRATION_INTERVAL == 0
            elite_before = population[1:div(POPULATION_SIZE, 2)]
            HTTP.WebSockets.open("ws://127.0.1.1:3333") do ws
                top_individuals = population[1:MIGRATION_COUNT]
                msg = Dict("type" => "migration", "from" => "worker_$id", "individuals"
                ↪  => top_individuals)
                HTTP.WebSockets.send(ws, JSON.json(msg))
                handle_messages(ws, population, NUM_WORKERS - 1)
            end
            population = vcat(elite_before, population)
            sort!(population, by=compute_fitness, rev=true)
            resize!(population, POPULATION_SIZE)
        end

        population = evolve_population!(population)
        decoded = [decode_chromosome(ch) for ch in population]
        zs = [evaluate_rosenbrock(x, y) for (x, y) in decoded]
        min_z = minimum(zs)

        if min_z < global_best[][3]
            idx = argmin(zs)
            best_x, best_y = decoded[idx][1], decoded[idx][2]
            global_best[] = (best_x, best_y, min_z)
        end

        lock(history_lock) do
            time_history[current_time] = min_z
            gen_history[generation] = min_z
        end
    end
end

function run_workers()
    pop = [create_random_chromosome() for _ in 1:(POPULATION_SIZE * NUM_WORKERS)]
    subpopulations = [pop[i:NUM_WORKERS:end] for i in 1:NUM_WORKERS]

    global_best = Ref((0.0, 0.0, Inf))
    time_history = Dict{Float64, Float64}()
    gen_history = Dict{Int, Float64}()
    history_lock = ReentrantLock()

    start_time = time()

    @sync for i in 1:NUM_WORKERS
        @spawn worker_process(i, subpopulations[i], time_history, gen_history,
        ↪  global_best, history_lock, start_time)
    end

    elapsed = (time() - start_time) * 1000
    println("Execution time: $(round(elapsed, digits=2)) milliseconds")

    best_x, best_y, best_f = global_best[]
    println("\nBest solution: f(x, y) = $(round(best_f, digits=5)) at x =
    ↪  $(round(best_x, digits=5)), y = $(round(best_y, digits=5))")
    return time_history, gen_history, best_f
end
```

```julia
function run_server()
    clients = Dict{String, HTTP.WebSockets.WebSocket}()

    function handle_client(ws)
        id = string(rand(UInt))
        clients[id] = ws

        while !HTTP.WebSockets.isclosed(ws)
            msg = String(HTTP.WebSockets.receive(ws))
            data = JSON.parse(msg)
            if data["type"] == "migration"
                println("Received migration from $(data["from"])")
                for (other_id, client_ws) in clients
                    if other_id != data["from"] && !HTTP.WebSockets.isclosed(client_ws)
                        HTTP.WebSockets.send(client_ws, JSON.json(data))
                    end
                end
            end
        end
    end

    HTTP.WebSockets.listen("127.0.1.1", 3333) do ws
        handle_client(ws)
    end
end

function run_sequential_evolution()
    population = [create_random_chromosome() for _ in 1:POPULATION_SIZE]
    global_best = Ref((0.0, 0.0, Inf))
    time_history = Dict{Float64, Float64}()
    gen_history = Dict{Int, Float64}()

    println("Starting sequential evolution...")
    start_time = time()

    for generation in 1:GENERATIONS
        current_time = (time() - start_time) * 1000
        population = evolve_population!(population)
        decoded = [decode_chromosome(ch) for ch in population]
        zs = [evaluate_rosenbrock(x, y) for (x, y) in decoded]
        min_z = minimum(zs)

        if min_z < global_best[][3]
            idx = argmin(zs)
            best_x, best_y = decoded[idx][1], decoded[idx][2]
            global_best[] = (best_x, best_y, min_z)
        end

        time_history[current_time] = min_z
        gen_history[generation] = min_z
    end

    elapsed = (time() - start_time) * 1000
    println("Execution time: $(round(elapsed, digits=2)) milliseconds")

    best_x, best_y, best_f = global_best[]
    println("\nBest solution: f(x, y) = $(round(best_f, digits=5)) at x =
    ↪ $(round(best_x, digits=5)), y = $(round(best_y, digits=5))")
    return time_history, gen_history, global_best[][3]
end

function plot_convergence(par_time_hist, par_gen_hist, seq_time_hist, seq_gen_hist)
    plt1 = plot(title="Convergence by Time (Rosenbrock)",
        xlabel="Time (milliseconds)", ylabel="Best Value",
```

```
        legend=:topright,
        yscale=:log10)

    plot!(plt1, sort(collect(keys(par_time_hist))), [par_time_hist[t] for t in
     ↪  sort(collect(keys(par_time_hist)))],
        label="Parallel", linewidth=2)

    plot!(plt1, sort(collect(keys(seq_time_hist))), [seq_time_hist[t] for t in
     ↪  sort(collect(keys(seq_time_hist)))],
        label="Sequential", linewidth=2)

    savefig(plt1, "time_convergence.png")

    plt2 = plot(title="Convergence by Generation (Rosenbrock)",
        xlabel="Generation", ylabel="Best Value",
        legend=:topright,
        yscale=:log10)

    plot!(plt2, sort(collect(keys(par_gen_hist))), [par_gen_hist[g] for g in
     ↪  sort(collect(keys(par_gen_hist)))],
        label="Parallel", linewidth=2)

    plot!(plt2, sort(collect(keys(seq_gen_hist))), [seq_gen_hist[g] for g in
     ↪  sort(collect(keys(seq_gen_hist)))],
        label="Sequential", linewidth=2)

    savefig(plt2, "generation_convergence.png")

    display(plt1)
    display(plt2)
end

function compare_algorithms()
    println("=== Parallel Algorithm ===")
    par_time_hist, par_gen_hist, best_par = run_workers()

    println("\n=== Sequential Algorithm ===")
    seq_time_hist, seq_gen_hist, best_seq = run_sequential_evolution()

    plot_convergence(par_time_hist, par_gen_hist, seq_time_hist, seq_gen_hist)

    println("\nFinal comparison:")
    println("Parallel best: ", round(best_par, digits=5))
    println("Sequential best: ", round(best_seq, digits=5))
end

if abspath(PROGRAM_FILE) == @__FILE__
    if length(ARGS) > 0 && ARGS[1] == "master"
        run_server()
    else
        compare_algorithms()
    end
end
```

Результаты

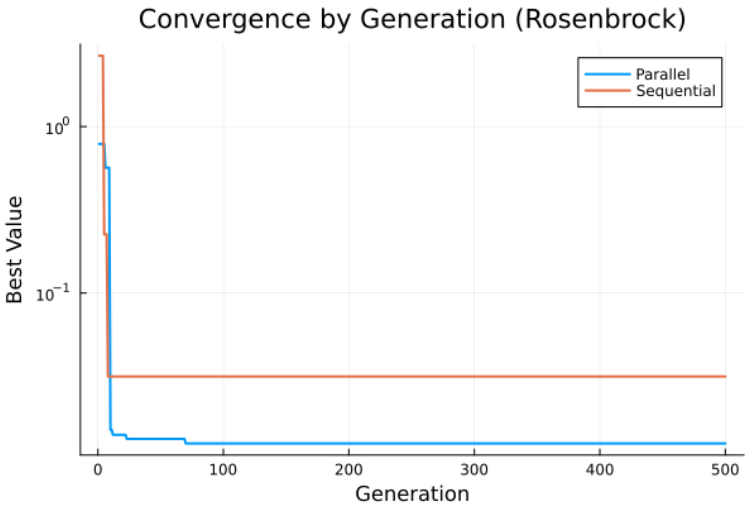

Рис. 1: результаты

Вывод

Параллельный генетический алгоритм с миграцией особей между подпопуляциями демонстрирует более быстрое достижение оптимального решения по сравнению с последовательной версией. Реализация на Julia с использованием многопоточности и WebSocket-коммуникации подтверждает эффективность параллельного подхода для сложных оптимизационных задач.